

Lab 8: Table Relationships — Product Shop

ส่วนที่ 1: หลักการออกแบบ

1. หลักการ SOLID ที่ใช้ในโปรเจกต์ และความเกี่ยวข้องกับ Entity

โปรเจกต์ Product Shop มีการออกแบบโดยใช้แนวคิดของ SOLID ในหลายส่วน โดยเฉพาะการแยกหน้าที่ของ Entity, Service, Controller และส่วนของการคำนวณส่วนลดออกจากกัน

1.1 Single Responsibility Principle (SRP)

หลักการ SRP กล่าวว่าแต่ละคลาสควรมีหน้าที่หลักที่ชัดเจนและไม่ควรรับผิดชอบหลายเรื่องเกินไป ในโปรเจกต์มีการแบ่งหน้าที่ของแต่ละส่วน ดังนี้

- Product, ProductDetail และ Review ทำหน้าที่เป็น Entity สำหรับเก็บข้อมูลและกำหนดความสัมพันธ์ระหว่างข้อมูล
- ProductService ทำหน้าที่เกี่ยวกับ Business Logic เช่น การเพิ่ม แก้ไข ลบ ค้นหาสินค้า และคำนวณราคาหลังหักส่วนลด
- ProductController ทำหน้าที่รับ HTTP Request และส่งข้อมูลระหว่างหน้าเว็บกับ Service
- DiscountStrategy และคลาสที่ implements ทำหน้าที่เกี่ยวกับวิธีการคำนวณส่วนลดแต่ละรูปแบบ

การแบ่งหน้าที่ดังกล่าวช่วยให้แต่ละส่วนของโปรแกรมมีความรับผิดชอบที่ชัดเจน และสามารถแก้ไขส่วนใดส่วนหนึ่งได้โดยไม่ต้องนำหน้าที่ทั้งหมดมาไว้ในคลาสเดียว

1.2 Open/Closed Principle (OCP)

หลักการ OCP กล่าวว่า Software ควรเปิดให้เพิ่มความสามารถใหม่ได้โดยไม่จำเป็นต้องแก้ไขส่วนที่เป็นโครงสร้างหลักเดิม ในโปรเจกต์มีการใช้ DiscountStrategy เป็น Interface สำหรับกำหนดรูปแบบการคำนวณส่วนลด และมีคลาสที่นำไปใช้งานได้แก่

- NoDiscountStrategy
- MemberDiscountStrategy

- SeasonalSaleStrategy

แต่ละคลาสสามารถมีวิธีคำนวณส่วนลดแตกต่างกันได้ โดยไม่ต้องเปลี่ยนแปลง Interface DiscountStrategy อย่างไรก็ตาม ในโค้ดปัจจุบัน ProductService.calculateFinalPrice() ยังใช้ if-else เพื่อเลือก Strategy ดังนั้นหากเพิ่มประเภทส่วนลดใหม่ จะต้องเพิ่มเงื่อนไขในเมธอดนี้ด้วย จึงถือว่าโปรเจกต์นำแนวคิด Strategy มาใช้เพื่อแยกส่วนของ Algorithm แต่ยังไม่ได้เป็น OCP แบบสมบูรณ์ทั้งหมด

1.3 Dependency Inversion Principle (DIP)

โปรเจกต์ใช้ Constructor Injection ใน ProductController และ ProductService เช่น

```
public ProductController(ProductService productService) {
```

```
    this.productService = productService; }
```

และ

```
public ProductService(ProductRepository productRepository){
```

```
    this.productRepository = productRepository; }
```

Dependency จะถูกส่งเข้ามาทาง Constructor แทนการสร้าง Object ภายในคลาสโดยตรง ทำให้ Spring สามารถจัดการ Dependency ให้กับคลาสเหล่านี้ได้ และช่วยลดการผูกกันของการสร้าง Object ภายในแต่ละคลาส

2. ความสัมพันธ์ระหว่าง Entity แบบ 1:1 และ 1

โปรเจกต์นี้มีการใช้ความสัมพันธ์ของ Entity อยู่ 2 รูปแบบ ได้แก่ 1:1 และ 1:N

2.1 ความสัมพันธ์แบบ 1:1 (One-to-One)

ความสัมพันธ์แบบ 1:1 หมายถึง ข้อมูลหนึ่งรายการใน Entity หนึ่งมีความสัมพันธ์กับข้อมูลได้หนึ่งรายการในอีก Entity หนึ่ง

ในโปรเจกต์ใช้ความสัมพันธ์นี้ระหว่าง Product 1 ————— 1 ProductDetail

สินค้าแต่ละรายการมีรายละเอียดสินค้า (ProductDetail) หนึ่งชุด และ ProductDetail แต่ละชุดมี Product ที่สัมพันธ์กันหนึ่งรายการ

- ใน Product ใช้

```
@OneToOne(cascade = CascadeType.ALL)
```

```
@JoinColumn(name = "detail_id", referencedColumnName = "id")
```

```
private ProductDetail detail;
```

- ส่วน ProductDetail ใช้

```
@OneToOne(mappedBy = "detail")
```

```
private Product product;
```

Product เป็นฝั่งที่เป็นเจ้าของความสัมพันธ์ เนื่องจากเป็นฝั่งที่มี @JoinColumn ส่วน ProductDetail ใช้ mappedBy เพื่อบอกว่าความสัมพันธ์ถูกกำหนดจาก Product

กรณีที่ใช้ 1:1 เหมาะกับข้อมูลที่มีความสัมพันธ์กันแบบหนึ่งต่อหนึ่ง เช่น

Product -> ProductDetail, User -> UserProfile, Employee -> EmployeeDetail โดยข้อมูลหนึ่งรายการต้องมีรายละเอียดอีกชุดที่สัมพันธ์กับมันโดยตรง

2.2 ความสัมพันธ์แบบ 1 (One-to-Many)

ความสัมพันธ์แบบ 1 หมายถึง ข้อมูลหนึ่งรายการสามารถมีข้อมูลอีกประเภทได้หลายรายการ

ในโปรเจกต์ใช้ความสัมพันธ์นี้ระหว่าง Product 1 ————— N Review

สินค้า 1 รายการสามารถมี Review ได้หลายรายการ

- ใน Product ใช้

```
@OneToMany(mappedBy = "product", cascade = CascadeType.ALL)
```

```
private List<Review> reviews = new ArrayList<>();
```

- ส่วน Review ใช้

```
@ManyToOne
```

```
@JoinColumn(name = "product_id", referencedColumnName = "id")
```

private Product product;

Review เป็นฝั่งเจ้าของความสัมพันธ์ เนื่องจากเป็นฝั่งที่มี @JoinColumn และเก็บ product_id ที่ใช้เชื่อมกับ Product

กรณีที่ใช้ 1 เหมาะกับกรณีที่ข้อมูลหลักหนึ่งรายการสามารถมีข้อมูลย่อยได้หลายรายการ เช่น Product -> Reviews, Customer -> Orders, Category -> Products

ดังนั้นในโปรเจกต์นี้ Product จึงใช้ 1:1 กับ ProductDetail เนื่องจากสินค้าหนึ่งรายการมีรายละเอียดสินค้าเป็นชุดเดียว และใช้ 1 กับ Review เนื่องจากสินค้าหนึ่งรายการสามารถมีรีวิวได้หลายรายการ

3. Strategy Pattern สำหรับคำนวณส่วนลด

โปรเจกต์ใช้ Strategy Pattern เพื่อแยกวิธีการคำนวณราคาสินค้าแต่ละรูปแบบออกจากกัน มี Interface หลักคือ

```
public interface DiscountStrategy {  
    double calculateDiscount(double price);  
}
```

จากนั้นมี Strategy ที่นำไปใช้งาน 3 แบบ ได้แก่

- NoDiscountStrategy -> ไม่มีส่วนลด ราคาที่ได้เท่ากับราคาปกติ

```
public class NoDiscountStrategy implements DiscountStrategy {  
    @Override  
    public double calculateDiscount(double price) {  
        return price;  
    }  
}
```

- MemberDiscountStrategy -> สมาชิกรับส่วนลด 10%

```
public class MemberDiscountStrategy implements DiscountStrategy {  
    @Override  
    public double calculateDiscount(double price) {  
        return price * 0.90;  
    }  
}
```

- SeasonalSaleStrategy -> ส่วนลด 20%

```
public class SeasonalSaleStrategy implements DiscountStrategy {  
    @Override  
    public double calculateDiscount(double price) {  
        return price * 0.80;  
    }  
}
```

- DiscountContext รับ Strategy และเรียกใช้การคำนวณ

```
public class DiscountContext {  
    private DiscountStrategy strategy;  
    public DiscountContext(DiscountStrategy strategy) {  
        this.strategy = strategy;  
    }  
    public double calculateFinalPrice(double price) {  
        return strategy.calculateDiscount(price);  
    }  
}
```

```

    }

    public void setStrategy(DiscountStrategy strategy) {

        this.strategy = strategy;

    }

}

```

- ProductService ตรวจสอบค่า discountType ของ Product แล้วเลือก Strategy ที่เหมาะสม

```

if("MEMBER".equalsIgnoreCase(product.getDiscountType())){

    context = new DiscountContext(new MemberDiscountStrategy());

} else if("SEASONAL".equalsIgnoreCase(product.getDiscountType())){

    context = new DiscountContext(new SeasonalSaleStrategy());

} else{

    context = new DiscountContext(new NoDiscountStrategy());
}

```

จากนั้นจึงเรียก return context.calculateFinalPrice(product.getPrice()); ข้อดีของการออกแบบนี้คือ วิธีคำนวณส่วนลดแต่ละประเภทถูกแยกออกเป็นคอนลลอส ทำให้โค้ดอ่านง่ายและสามารถเพิ่ม Strategy ใหม่ได้โดยไม่ต้องเปลี่ยน Interface เดิม

4. Execution Flow ตั้งแต่ HTTP Request -> DB

การทำงานของโปรเจกต์แบ่งเป็นลำดับหลักๆ ดังนี้ Browser -> ProductController -> ProductService -> ProductRepository -> Database

4.1 การแสดงรายการสินค้า

เมื่อผู้ใช้เรียกหน้า GET /products, ProductController จะเรียก productService.getAllProducts(); จากนั้น ProductService เรียก productRepository.findAll(); เพื่อดึงข้อมูล Product จาก Database เมื่อได้รายการสินค้า

แล้ว Controller จะคำนวณราคาหลังหักส่วนลดของแต่ละสินค้า และเก็บไว้ใน discountedPrices โดยใช้ Product ID เป็น Key จากนั้นส่งข้อมูล
model.addAttribute("products", products);

model.addAttribute("discountedPrices", discountedPrices); ไปยัง Thymeleaf หน้า products/list

4.2 การเพิ่มสินค้า

เมื่อผู้ใช้เปิดหน้าเพิ่มสินค้า จะเรียก GET /products/add , Controller สร้าง Product ใหม่ และส่งไปยังหน้า products/add เมื่อผู้ใช้กดบันทึก จะส่ง POST /products/save เข้ามาที่ Controller ก่อนบันทึก Controller จะตรวจสอบ ProductDetail และกำหนด Product ให้กับ Detail

```
if (product.getDetail() != null) {  
    product.getDetail().setProduct(product);  
}
```

จากนั้นเรียก productService.saveProduct(product); และ Service จะเรียก productRepository.save(product); เพื่อบันทึกข้อมูลลง Database

4.3 การแก้ไขสินค้า

เมื่อเรียก GET /products/edit/{id} , Controller จะค้นหา Product จาก ID ผ่าน Service
productService.getProductById(id)

Service จะเรียก productRepository.findById(id) เมื่อผู้ใช้ส่งข้อมูลแก้ไขผ่าน POST /products/update/{id}

Controller จะเรียก productService.updateProduct(id, product);

Service จะค้นหา Product เดิม จากนั้นแก้ไขข้อมูล เช่น name, category, brand, stock, price, discountType และหากมี ProductDetail ก็จะอัปเดตข้อมูลรายละเอียดสินค้า จากนั้นเรียก productRepository.save(existingProduct) เพื่อบันทึกข้อมูล

ส่วนที่ 2: Code + คำอธิบาย

1. Entity

โปรเจกต์มี Entity หลัก 3 ตัว ได้แก่ Product, ProductDetail, Review โดย Entity แต่ละตัวแทนข้อมูลที่จัดเก็บใน Database และใช้ Annotation ของ JPA เพื่อกำหนดตารางและความสัมพันธ์ระหว่างข้อมูล

1.1 Product.java

@Entity

@Table(name = "products")

public class Product {

 @Id

 @GeneratedValue(strategy = GenerationType.IDENTITY)

 private Long id;

 private String name;

 private String category;

 private String brand;

 private int stock;

 private double price;

 private String discountType;

 @OneToOne(cascade = CascadeType.ALL)

 @JoinColumn(name = "detail_id", referencedColumnName = "id")


```
private ProductDetail detail;
```

```
@OneToMany(mappedBy = "product", cascade = CascadeType.ALL)
```

```
private List<Review> reviews = new ArrayList<>();}
```

คำอธิบาย Annotation

@Entity -> กำหนดให้คลาส Product เป็น JPA Entity ซึ่งสามารถถูกนำไปเชื่อมโยงกับข้อมูลใน Database ได้

@Table(name = "products") -> กำหนดให้ Entity นี้เชื่อมโยงกับตารางชื่อ products

@Id -> กำหนดให้ id เป็น Primary Key ของ Product

@GeneratedValue(strategy = GenerationType.IDENTITY) -> กำหนดให้ค่าของ ID ถูกสร้างโดย Database ตามรูปแบบ Identity ดังนั้น ID ใน Database เป็น ID สำหรับระบุข้อมูลแต่ละรายการ ไม่ใช่หมายเลขลำดับที่ใช้แสดงในหน้าเว็บ และเมื่อมีการลบข้อมูล ID เดิมจะไม่ถูกนำกลับมาเรียงใหม่โดยอัตโนมัติ

@OneToOne(cascade = CascadeType.ALL) -> กำหนดความสัมพันธ์แบบ 1:1 ระหว่าง Product กับ ProductDetail และกำหนด Cascade ให้การดำเนินการกับ Product สามารถส่งต่อไปยัง ProductDetail ได้

@JoinColumn(name = "detail_id", referencedColumnName = "id") -> กำหนด Column detail_id สำหรับเชื่อม Product กับ ProductDetail โดยอ้างอิงไปยัง id ของ ProductDetail

@OneToMany(mappedBy = "product", cascade = CascadeType.ALL) -> กำหนดความสัมพันธ์แบบ 1 ระหว่าง Product กับ Review โดย Product หนึ่งรายการสามารถมี Review ได้หลายรายการ mappedBy = "product" หมายถึงความสัมพันธ์นี้ถูกกำหนดโดย field product ใน Entity Review

1.2 ProductDetail.java

```
@Entity
```

```
@Table(name = "product_details")
```

```
public class ProductDetail {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
    private String description;  
    private String warranty;  
    private Double weight;  
    private String dimensions;  
    private String manufacturedCountry;  
    @OneToOne(mappedBy = "detail")  
    private Product product;  
}
```

คำอธิบาย Annotation

@Entity -> กำหนดให้ ProductDetail เป็น JPA Entity

@Table(name = "product_details") -> กำหนดให้ข้อมูลของ Entity นี้เชื่อมโยงกับตาราง product_details

@Id -> กำหนด id เป็น Primary Key ของ ProductDetail

@GeneratedValue(strategy = GenerationType.IDENTITY) -> กำหนดให้ Database สร้างค่า ID ให้อัตโนมัติ

@OneToOne(mappedBy = "detail") -> กำหนดความสัมพันธ์แบบ 1:1 กับ Product

mappedBy = "detail" หมายถึงฝั่ง ProductDetail ไม่ได้เป็นเจ้าของความสัมพันธ์ แต่ให้ Product เป็นผู้กำหนดความสัมพันธ์ผ่าน field detail ดังนั้นความสัมพันธ์จึงเป็น Product.detail <-> ProductDetail.product

1.3 Review.java

@Entity

@Table(name = "reviews")

public class Review {

 @Id

 @GeneratedValue(strategy = GenerationType.IDENTITY)

 private Long id;

 private String reviewer;

 private int rating;

 private String comment;

 private LocalDate reviewDate;

 @ManyToOne

 @JoinColumn(name = "product_id", referencedColumnName = "id")

 private Product product;

}

คำอธิบาย Annotation

@Entity -> กำหนดให้ Review เป็น JPA Entity

@Table(name = "reviews") -> กำหนดให้ Entity นี้เชื่อมโยงกับตาราง reviews

@Id -> กำหนด id เป็น Primary Key ของ Review

@GeneratedValue(strategy = GenerationType.IDENTITY) -> กำหนดให้ Database สร้าง ID ของ Review ให้อัตโนมัติ

@ManyToOne -> กำหนดความสัมพันธ์แบบหลายต่อหนึ่ง โดย Review หลายรายการสามารถสัมพันธ์กับ Product หนึ่งรายการได้

ตัวอย่างเช่น

Product A

└── Review 1

└── Review 2

└── Review 3

@JoinColumn(name = "product_id", referencedColumnName = "id") -> กำหนด Column product_id ใน Review เพื่อใช้เชื่อมกับ id ของ Product ดังนั้น Review เป็นฝั่งที่เป็นเจ้าของความสัมพันธ์ระหว่าง Product และ Review

2. ProductService.java

ProductService ทำหน้าที่เป็นชั้น Business Logic ระหว่าง Controller กับ Repository

โครงสร้าง Dependency คือ

ProductController -> ProductService -> ProductRepository -> Database

โค้ดส่วนสำคัญของ Service คือ

@Service

```
public class ProductService {  
  
    private final ProductRepository productRepository;  
  
    public ProductService(ProductRepository productRepository){  
  
        this.productRepository = productRepository;  
  
    }  
  
    public List<Product> getAllProducts(){  
  
        return productRepository.findAll();  
  
    }  
}
```

```
public Optional<Product> getProductById(Long id){  
    return productRepository.findById(id);  
}  
  
public Product saveProduct(Product product){  
    return productRepository.save(product);  
}  
  
public void deleteProduct(Long id){  
    productRepository.deleteById(id);  
}  
}
```

คำอธิบาย

@Service -> กำหนดให้ ProductService เป็น Spring Service ซึ่งใช้สำหรับจัดการ Business Logic ของระบบ

private final ProductRepository productRepository -> เป็น Dependency ที่ Service ต้องใช้สำหรับติดต่อกับข้อมูล Product

getAllProducts() -> เรียก productRepository.findAll() เพื่อดึง Product ทั้งหมดจาก Database

getProductById(Long id) -> เรียก findById(id) เพื่อค้นหา Product จาก ID และคืนค่าเป็น Optional<Product>

saveProduct(Product product) -> เรียก save() เพื่อบันทึกหรือจัดเก็บ Product

deleteProduct(Long id) -> เรียก deleteById(id) เพื่อลบ Product ตาม ID นอกจากนี้ ProductService ยังมี updateProduct() สำหรับแก้ไขข้อมูล Product และ calculateFinalPrice() สำหรับคำนวณราคาหลังหักส่วนลด

ส่วนที่ 3: ภาพหน้าจอ

3.1.1 เพิ่มรายการสินค้าใหม่

← → ↺ ⓘ localhost:8080/products/add

+ เพิ่มสินค้าใหม่

📦 ข้อมูลสินค้า (Product)

ชื่อสินค้า

Logitech MX Master 3S

หมวดหมู่

Computer Accessories

ยี่ห้อ

Logitech

จำนวนในคลัง

84

ราคาปกติ (บาท)

3290.0

ประเภทส่วนลด

ไม่มีส่วนลด

📋 ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า

การเชื่อมต่อหลายอุปกรณ์และสามารถปรับความละเอียดของเซ็นเซอร์ได้

ระยะเวลาประกัน

2 Year

น้ำหนัก (กิโลกรัม)

0.141

ขนาด (กว้าง x ยาว x สูง)

12.5x8.4x5.1 cm

ประเทศที่ผลิต

China

★ รีวิวสินค้า (Review) — ความสัมพันธ์ 1:N

เพิ่มรีวิวแรกพร้อมกับสินค้า (สามารถเพิ่มรีวิวเพิ่มเติมภายหลังได้)

ชื่อผู้รีวิว

อิมสบาย นอนหลับปุ๋ย

คะแนน (1-5)

★★★★ (4)

ความเห็น

เมาส์ไต่ลื่น และเหมาะกับการทำงานเป็นเวลานาน

📄 บันทึกสินค้า

ยกเลิก

3.1.2 หลังเพิ่มสินค้าเข้ารายการ

localhost:8080/products

Product Shop Lab 8 — 1:1 & 1:N Relationship

+ เพิ่มสินค้าใหม่

#	ชื่อสินค้า	หมวดหมู่	ยี่ห้อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	รับประกัน (1:1)	น้ำหนัก (1:1)	รีวิว (1:N)	จัดการ
1	iPhone 17 (673380289-4 SEC1)	Electronic	Apple	124	29900.00	MEMBER	26910.00	3 Year	0.17	0 รีวิว	แก้ไข ลบ
2	Tablet s6	Electronic	Samsung	9	17000.00	SEASONAL	13600.00	1 Year	1.61	0 รีวิว	แก้ไข ลบ
3	AirPods Pro 3 (673380289-4 SEC1)	Electronic	Apple	50	8490.00	NONE	8490.00	1 Year	0.05	0 รีวิว	แก้ไข ลบ
4	Logitech MX Master 3S (673380289-4 SEC1)	Computer Accessories	Logitech	84	3290.00	NONE	3290.00	2 Year	0.14	0 รีวิว	แก้ไข ลบ

3.2.1 แก้ไขสินค้า

localhost:8080/products/edit/8

แก้ไขสินค้า

ข้อมูลสินค้า (Product)

ชื่อสินค้า
 หมวดหมู่
 ยี่ห้อ
 จำนวนในคลัง
 ราคาปกติ (บาท)
 ประเภทส่วนลด

ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า
 ระยะเวลาประกัน
 น้ำหนัก (กิโลกรัม)
 ขนาด
 ประเทศที่ผลิต
 บันทึกการแก้ไข [ยกเลิก](#)

3.2.2 หลังแก้ไขสินค้า

← → ↻ ⓘ localhost:8080/products

Product Shop Lab 8 — 1:1 & 1:N Relationship

[+ เพิ่มสินค้าใหม่](#)

#	ชื่อสินค้า	หมวดหมู่	ยี่ห้อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	รับประกัน (1:1)	น้ำหนัก (1:1)	รีวิว (1:N)	จัดการ
1	iPhone 17 (673380289-4 SEC1)	Electronic	Apple	124	29900.00	MEMBER	26910.00	3 Year	0.17	0 รีวิว	แก้ไข ลบ
2	AirPods Pro 3 (673380289-4 SEC1)	Electronic	Apple	50	8490.00	NONE	8490.00	1 Year	0.05	0 รีวิว	แก้ไข ลบ
3	Logitech MX Master 3S (673380289-4 SEC1)	Computer Accessories	Logitech	84	3290.00	NONE	3290.00	2 Year	0.14	0 รีวิว	แก้ไข ลบ
4	Tablet s6(673380289-4 SEC1)	Electronic	Samsung	9	17000.00	SEASONAL	13600.00	1 Year	1.61	0 รีวิว	แก้ไข ลบ

3.3.1 ลบสินค้า

← → ↻ ⓘ localhost:8080/products/delete/9

ยืนยันการลบสินค้า

คุณต้องการลบสินค้านี้ใช่ไหม? การดำเนินการนี้ไม่สามารถย้อนกลับได้

ชื่อสินค้า: AirPods Pro 3 (673380289-4 SEC1)

หมวดหมู่: Electronic

ยี่ห้อ: Apple

ราคา: 8490.00 บาท

รายละเอียด: หูฟังไร้สายพร้อมระบบตัดเสียงรบกวน ให้เสียงคมชัด เหมาะสำหรับฟังเพลง ดูหนัง และใช้งานระหว่างเดินทาง

รับประกัน: 1 Year

 ข้อมูลเสริม (ProductDetail) จะถูกลบพร้อมกันด้วย เนื่องจาก CascadeType.ALL

ยืนยันลบ [ยกเลิก](#)

3.3.2 หลังลบสินค้า

 Product Shop Lab 8 — 1:1 & 1:N Relationship

[+ เพิ่มสินค้าใหม่](#)

#	ชื่อสินค้า	หมวดหมู่	ยี่ห้อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	รับประกัน (1:1)	น้ำหนัก (1:1)	รีวิว (1:N)	จัดการ
1	iPhone 17 (673380289-4 SEC1)	Electronic	Apple	124	29900.00	MEMBER	26910.00	3 Year	0.17	0 รีวิว	แก้ไข ลบ
2	Logitech MX Master 3S (673380289-4 SEC1)	Computer Accessories	Logitech	84	3290.00	NONE	3290.00	2 Year	0.14	0 รีวิว	แก้ไข ลบ
3	Tablet s6(673380289-4 SEC1)	Electronic	Samsung	9	17000.00	SEASONAL	13600.00	1 Year	1.61	0 รีวิว	แก้ไข ลบ